

Lab 2: Reverse Engineering & Binary Patching

Author: Diana Yakupova

Email: d.yakupova@innopolis.university

Group: B23-CBS-02

GitHub link: <https://github.com/versceana/advanced-linux/tree/main/lab2-gdb>

Task

Study GDB, strace, ldd and related utilities. “Hack” the instructor-provided application that ties a license key to a hardware ID (HWID). The license is persisted somewhere on the filesystem.

Required artifacts:

1. **Keygen** – generates valid license keys for arbitrary HWIDs.
 2. **Binary patch** – modifies the executable so that license checks are disabled.
-

Implementation steps

Step 1 – Environment preparation

The target binary depends on `libcrypto.so.1.1` from OpenSSL 1.1.

Ubuntu 24.04 ships OpenSSL 3, so I manually installed the older package:

```
wget https://security.debian.org/debian-security/pool/updates/main/o/openssl/libssl1.1_1.1.1w-0+deb11u5_amd64.deb
sudo dpkg -i libssl1.1_1.1.1w-0+deb11u5_amd64.deb
```

I also installed the `attr` package to inspect extended attributes later:

```
sudo apt install -y attr
```

After that `ldd ./hack_app` showed all dependencies resolved.

```
ubuntu@ip-172-31-30-209 ~ (0.261s)
wget https://security.debian.org/debian-security/pool/updates/main/o/openssl/libssl1.1_1.1.1w-0+deb11u5_amd64.deb
--2026-05-10 09:49:53-- https://security.debian.org/debian-security/pool/updates/main/o/openssl/libssl1.1_1.1.1w-0+deb11u5_amd64.deb
Resolving security.debian.org (security.debian.org)... 151.101.2.132, 151.101.130.132, 151.101.66.132, ...
Connecting to security.debian.org (security.debian.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1564976 (1.5M) [application/vnd.debian.binary-package]
Saving to: 'libssl1.1_1.1.1w-0+deb11u5_amd64.deb'

libssl1.1_1.1.1w-0+deb11u5_amd64.deb      100%[=====] 1.49M  --.-KB/s  in 0.01s

2026-05-10 09:49:53 (150 MB/s) - 'libssl1.1_1.1.1w-0+deb11u5_amd64.deb' saved [1564976/1564976]

ubuntu@ip-172-31-30-209 ~ (0.693s)
sudo dpkg -i libssl1.1_1.1.1w-0+deb11u5_amd64.deb
Selecting previously unselected package libssl1.1:amd64.
(Reading database ... 80831 files and directories currently installed.)
Preparing to unpack libssl1.1_1.1.1w-0+deb11u5_amd64.deb ...
Unpacking libssl1.1:amd64 (1.1.1w-0+deb11u5) ...
Setting up libssl1.1:amd64 (1.1.1w-0+deb11u5) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...

ubuntu@ip-172-31-30-209 ~ (1.635s)
find /usr/lib -name "libcrypto.so.1.1"
/usr/lib/x86_64-linux-gnu/libcrypto.so.1.1
```

Step 2 – Initial triage

I gathered basic information about the binary:

```
file ./hack_app      # ELF 64-bit LSB pie executable, x86-64, not stripped
ldd ./hack_app       # depends on libcrypto.so.1.1, libc.so.6, ...
strings ./hack_app   # revealed "user.license", MD5 functions, HWID format
"%08X%08X"
```

Running `strace` showed exactly how the program locates itself and accesses the license:

```
strace -o trace.txt ./hack_app
grep -E 'getxattr|setxattr|readlink' trace.txt
# readlink("/proc/self/exe", ...) → path to binary
# getxattr(".../hack_app", "user.license", ...) → ENODATA when no license
present
```

This proved that the license is stored as the extended attribute `user.license` of the executable itself, a nice alternative to a separate license file.

```
ubuntu@ip-172-31-30-209 ~/lab2 (0.179s)
file ./hack_app
./hack_app: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=0813fa48181874628c171f5ed6f0f48b6af0d844, for GNU/Linux 3.2.0, not stripped

ubuntu@ip-172-31-30-209 ~/lab2 (0.216s)
ldd ./hack_app
linux-vdso.so.1 (0x000078bae0d1000)
libcrypto.so.1.1 => /lib/x86_64-linux-gnu/libcrypto.so.1.1 (0x000078bae0a00000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000078bae0600000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x000078bae0d50000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x000078bae0d00000)
/lib64/ld-linux-x86-64.so.2 (0x000078bae0d21000)
```

[illegible]

By reading the assembly I reconstructed the license generation logic step by step:

1. **CPUID** – the instruction `call __get_cpuid` retrieves four 32-bit values. Only `eax` and `edx` are kept.
2. **Byte swapping** – each of those two values is byte-swapped with shifts and masks, yielding two human-readable 32-bit words.
3. **HWID string** – `snprintf` formats them as 16 uppercase hex characters (`"%08X%08X"`), e.g. `54060500FFFB8B1F` .
4. **MD5** – the string is passed to `calc_md5` , which internally calls `MD5_Init` , `MD5_Update` , `MD5_Final` . The 16-byte binary digest lands in `md5digest` .
5. **License key string** – a loop with `sprintf` and a pointer to `md5decode` iterates 16 times. It reads bytes from `md5digest` in *reverse* order and formats each as two lowercase hex digits. The result is the 32-character license key.
6. **Verification flow:**
 - First, `getxattr` tries to read `user.license` . If it exists and equals the freshly computed key, the program prints “Your app is licensed...” and exits.
 - Otherwise it prompts the user, compares the input with the computed key, and on success stores the key via `setxattr` . On failure it prints an error and terminates.

Understanding this logic was crucial for both the keygen and the patch.

```

ubuntu@ip-172-31-30-209: ~/lab2 (15m 41.80s)
gdb ./hack_app
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./hack_app...
(No debugging symbols found in ./hack_app)
(gdb) break main
Breakpoint 1 at 0x13ec
(gdb) run
Starting program: /home/ubuntu/lab2/hack_app
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x00005555555553e4 in main ()
(gdb) disassemble main
Dump of assembler code for function main:
0x00005555555553e4 <+0>:    endbr64
0x00005555555553e8 <+4>:    push    %rbp
0x00005555555553e9 <+5>:    mov     %rsp,%rbp
=> 0x00005555555553ec <+8>:    sub     $0x40,%rsp
0x00005555555553f0 <+12>:   mov     %edi,-0x34(%rbp)
0x00005555555553f3 <+15>:   mov     %rsi,-0x40(%rbp)
0x00005555555553f7 <+19>:   mov     %fs:0x28,%rax
0x0000555555555400 <+28>:   mov     %rax,-0x8(%rbp)
0x0000555555555404 <+32>:   xor     %eax,%eax
0x0000555555555406 <+34>:   movl    $0x1,-0x14(%rbp)
0x000055555555540d <+41>:   movl    $0x3,-0x2c(%rbp)
0x0000555555555414 <+48>:   movl    $0x0,-0x28(%rbp)
0x000055555555541b <+55>:   movl    $0x0,-0x24(%rbp)
0x0000555555555422 <+62>:   movl    $0x0,-0x20(%rbp)
0x0000555555555429 <+69>:   movl    $0x0,-0x10(%rbp)
0x0000555555555430 <+76>:   movl    $0x0,-0xc(%rbp)
0x0000555555555437 <+83>:   movl    $0x0,-0x1c(%rbp)
0x000055555555543e <+90>:   lea     -0x20(%rbp),%rdi
0x0000555555555442 <+94>:   lea     -0x24(%rbp),%rcx
0x0000555555555446 <+98>:   lea     -0x28(%rbp),%rdx
0x000055555555544a <+102>:  lea     -0x2c(%rbp),%rsi
0x000055555555544e <+106>:  mov     -0x14(%rbp),%eax
0x0000555555555451 <+109>:  mov     %rdi,%r8
0x0000555555555454 <+112>:  mov     %eax,%edi
0x0000555555555456 <+114>:  call    0x55555555531e <__get_cpuid>
0x000055555555545b <+119>:  mov     -0x2c(%rbp),%eax
0x000055555555545e <+122>:  shr     %x18,%eax
0x0000555555555461 <+125>:  mov     %eax,%edx

```

Step 4 – Extracting the licence key with GDB

Because the binary is not stripped, symbol names like `md5decode` are visible. I placed a breakpoint right before the `strncmp` that checks the user's input. From the disassembly that call is at `main+0x211` (file offset `0x15f5`).

```
(gdb) break *main+0x211
(gdb) run
# Enter dummy key: aaaa
# Breakpoint hits
(gdb) x/s $rdi
0x555555559040 <md5decode>: "712dd8557397b440cf53db8338f1f5fe"
```

The `$rdi` register contained the address of the pre-computed correct key. I copied it and exited GDB.

```
(gdb) break *main+0x211
Breakpoint 1 at 0x15f5
(gdb) run
Starting program: /home/ubuntu/lab2/hack_app
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
Downloading separate debug info for /lib/x86_64-linux-gnu/libcrypto.so.1.1
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Welcome to Lab2 super secure program!
Your HWID is 54060500FFB881F.
Enter the license key: aaaa

Breakpoint 1, 0x00005555555555f5 in main ()
(gdb) x/s $rdi
0x555555559040 <md5decode>: "712dd8557397b440cf53db8338f1f5fe"
(gdb) exit
```

Step 5 – Activating the application

I ran the program normally and supplied the extracted key:

```
./hack_app
# Enter: 712dd8557397b440cf53db8338f1f5fe
# Output: Now you app is activated! Thanks for purchasing!
```

A quick check confirmed the license was saved:

```
getfattr -n user.license ./hack_app
# user.license="712dd8557397b440cf53db8338f1f5fe"
```

From that point on, every launch printed “Your app is licensed to this PC!”

```
ubuntu@ip-172-31-30-200 ~/lab2 (7.384s)
./hack_app
Welcome to Lab2 super secure program!
Your HWID is 54060500FFB881F.
Enter the license key: 712dd8557397b440cf53db8338f1f5fe
Now you app is activated! Thanks for purchasing!
Press Enter to continue...

ubuntu@ip-172-31-30-200 ~/lab2 (0.169s)
getfattr -n user.license ./hack_app
# file: hack_app
user.license="712dd8557397b440cf53db8338f1f5fe"
```

Step 6 – Keygen (`src/keygen.py`)

Armed with the algorithm, I wrote a Python key generator.

It takes a 16-character HWID, computes its MD5 hash, reverses the 16-byte digest, and formats each byte as two lowercase hex characters - exactly mirroring the loop found in the disassembly.

```
#!/usr/bin/env python3
import sys, hashlib

if len(sys.argv) != 2:
    print(f"Usage: {sys.argv[0]} <HWID>")
    sys.exit(1)

hwid = sys.argv[1]
digest = hashlib.md5(hwid.encode()).digest()
key = ''.join(f'{b:02x}' for b in reversed(digest))
print(key)
```

Testing it with the same HWID gave the identical key I had extracted earlier:

```
$ python3 keygen.py 54060500FFFB8B1F
712dd8557397b440cf53db8338f1f5fe
```

The keygen therefore works for any HWID. It requires no external libraries besides Python's standard `hashlib`.

```
ubuntu@ip-172-31-38-289 ~/lab2 (0.193s)
python3 keygen.py 54060500FFFB8B1F
712dd8557397b440cf53db8338f1f5fe
```

Step 7 – Binary patch (`src/patch_license.py`)

To permanently bypass the license check I decided to patch the conditional jump that follows the user-input comparison.

From the disassembly:

```
15f5:  call    1150 <strcmp@plt>
15fc:  jne     1631 <main+0x24d>    ; jump to failure message
```

The two bytes `75 33` (`jne` with a relative offset) can be replaced with `EB 00` – a short unconditional jump that always lands on the next instruction, which is the beginning of the success handler.

I wrote a small Python script that opens the binary, verifies the original bytes and overwrites them:

```

offset = 0x15fc
orig = b'\x75\x33'
patch = b'\xEB\x00'

with open('hack_app', 'r+b') as f:
    f.seek(offset)
    data = f.read(2)
    if data != orig:
        print(f'ERROR: unexpected bytes at {offset:#x}')
        exit(1)
    f.seek(offset)
    f.write(patch)
    print(f'Patched: {orig.hex()} → {patch.hex()} at {offset:#x}')

```

After applying the patch and clearing any existing license attribute, I ran the binary and entered a deliberately wrong key:

```

setfattr -x user.license ./hack_app
./hack_app
# Enter: bbbb
# Output: Now you app is activated! Thanks for purchasing!

```

The patch successfully disabled the protection. The program now accepts anything as a valid license.

```

ubuntu@ip-172-31-38-209 ~/lab2 (0.165s)
setfattr -x user.license ./hack_app

ubuntu@ip-172-31-38-209 ~/lab2 (7.457s)
./hack_app
Welcome to Lab2 super secure program!
Your HWID is 54060500FFB8B1F.
Enter the license key: bbbb
Now you app is activated! Thanks for purchasing!
Press Enter to continue...

```

Requirements checklist

Requirement	Status
GDB, strace, ldd used for analysis	Done
License algorithm fully recovered	Done
Working keygen for any HWID	Done
Binary patch that disables license	Done
Report with screenshots	Done

Key technical decisions

- **Breakpoint on the `call` instruction, not inside `libc`** - avoids pending breakpoint problems and PIE variability.
- **Offset-based breakpoint (`main+0x211`)** - works regardless of ASLR.
- **Direct file-offset patching** - `objdump` provided the file offset, so the patcher doesn't need to parse ELF headers.
- **Keygen follows the exact original transformation** - including digest reversal and lowercase formatting, ensuring 100% compatibility.
- **Clean separation** - source code, artifacts, and screenshots are organised in dedicated directories, as expected on a Linux system.

Conclusion

Both required artifacts were implemented and tested. The license algorithm was fully recovered using static and dynamic analysis tools. The keygen reliably generates valid keys for any HWID, and the binary patch permanently disables the license verification. All work was carried out on Ubuntu 24.04 x86_64, matching the target environment.